NATIONAL AUTONOMOUS UNIVERSITY OF MEXICO

FACULTY OF ENGINEERING

COMPUTER GRAPHICS AND HUMAN-COMPUTER INTERACTION

SEMESTER 2027 - 1

TEXTURING, COLOR, AND LIGHTING

STUDENTS	*Basilio Illescas Andrés*	*Product Owner*
	Trejo Olvera Emmanuel	*Scrum Master*
	Pérez Olvera Alexis Abraham	*Developer*
	Cruz Macedo Samuel Santiago	*Developer*
PROFESSOR	*Eng. José Ramón Pérez Athié*	
GROUP	*04*	

Ciudad Universitaria, CDMX

October 28, 2026

Contents

1 Introduction **2**

2 Objectives **2**

2.1 General Objective 2

2.2 Specific Objectives 2

3 Scope of the Delivery **2**

4 Agile Workflow Methodology **3**

5 Design and Development **3**

5.1 Blender Workflow: Procedural Textures 3

5.2 Normal Mapping and PBR Materials 3

5.3 Procedural Generation of UVs and Alpha Clipping 4

5.4 Hybrid Lighting and Transmission 4

5.5 Atmospheric Environment (Skybox) 4

6 Architectural Challenges and Cross-Platform Compatibility **4**

7 Project Structure **5**

8 Application Screenshots **6**

9 Compilation and Execution **9**

9.1 Launchers by Platform 9

10 Results and Conclusions **9**

11 References **10**

1 Introduction

After consolidating the geometric modeling base in the previous stage, the project advances towards the application of optical and lighting properties to bring realism to the Day of the Dead scene. It should be noted that most of our custom models were remodeled for this phase, demonstrating a greater commitment to the project and significantly increasing the visual and geometric quality of this delivery.

In this stage, the gray models are transformed through the application of texture maps (Base Color, Roughness, and Normals), the configuration of transparent materials, the creation of an immersive environment (*Skybox*), and the implementation of a dynamic lighting system.

Development was maintained in C++17 using OpenGL 3.3. Specific *shaders* were programmed to process simplified PBR textures and light the scene using a hybrid model that combines ambient and directional light (Moon) with point lights (candles and lanterns) and quadratic attenuation. All the work was structured to maintain optimal performance while processing thousands of textured instances simultaneously.

2 Objectives

2.1 General Objective

Implement a real-time texturing, materials, and lighting system over the previously modeled scene, achieving an immersive visual finish that respects geometric proportions and optimizes resources in a cross-platform environment.

2.2 Specific Objectives

- Design and bake PBR textures (Color, Normals, and Roughness) from procedural nodes in Blender.
- Implement dynamic 2D image loading from the user's file system, employing native dependencies wherever possible.
- Configure a *Skybox* with panoramic textures to simulate the lighting transition of a sunset.
- Develop *shaders* that calculate point lights with attenuation and simulated optical refraction for glass and liquids.
- Guarantee the correct compilation and execution of libraries and executables across Arch Linux, Linux Mint, macOS, and Windows 10.

3 Scope of the Delivery

The scope of this phase includes the texturing of the 18 custom models, the application of materials to the library models, the procedural creation of UV coordinates, transparency mapping (alpha clipping), and the programming of lighting *shaders*.

This delivery includes diffuse maps, normal maps calculated in real-time through mathematical derivatives of the fragments, metalness/roughness channels, and the cubic atmospheric environment. It also

incorporates simulated optical refraction through transmittance calculations and the *Fresnel* effect for vases and glasses.

Complex skeletal animations, high-resolution dynamic shadows (*Shadow Mapping*), and real-time ray-traced specular reflections remain outside the scope of this delivery.

4 Agile Workflow Methodology

To manage this second stage of the project, the team continued iterating on the *Scrum* framework, adapting the *sprints* to the specific needs of UV mapping, *shader* programming, and performance testing.

The *Product Backlog* was restructured to focus on the visual finish and optimization. Development was divided into four main iterations (*sprints*):

1. **Definition and Procedural Design:** Assignment of material properties, creation of node networks in Blender, and baking of PBR textures.
2. **Graphics Programming:** Development and integration of *shaders* for hybrid lighting, real-time normal calculations, and optical transmission.
3. **Integration and Cross-Platform Testing:** Creation of native routines for image loading, *Skybox* validation, and resolution of deployment conflicts on macOS, Linux, and Windows.
4. **Optimization and Refactoring:** Identification of lighting bottlenecks, geometric retopology of models with excessive polygon counts, and automation via native launchers.

Asynchronous sync meetings allowed us to early identify architectural roadblocks (such as Git repository conflicts due to OS-specific configurations), facilitating quick decision-making by the *Scrum Master* and the developers to unify the `CMakeLists.txt` and keep the delivery within the established deadlines.

5 Design and Development

5.1 Blender Workflow: Procedural Textures

To guarantee high visual quality without relying on external photographs, a procedural workflow was chosen. The textures of our custom models were designed directly in Blender's node editor using mathematical noise functions and color blending.

Subsequently, these complex node networks were baked into traditional bitmaps (Color, Roughness, and Normals). Finally, the models, along with their textures, were exported under the glTF 2.0 standard and stored in the `assets/models/propios/` path. This packaging ensured highly efficient reading when loading the scene in OpenGL.

5.2 Normal Mapping and PBR Materials

Instead of loading and precalculating tangent vectors in the vertices, the *fragment shader* (`escena.frag`) calculates the mapped normals in real-time. This was achieved by using the `dFdx` and `dFdy` functions

over the vertex positions and UV coordinates, allowing us to extract highly detailed reliefs for elements such as the *pan de muerto*, adobe walls, and calaveritas in a highly optimized manner.

5.3 Procedural Generation of UVs and Alpha Clipping

For the cobblestone path and the cempasuchil petal carpet, UV coordinates were generated procedurally in the `Camino.cpp` file based on the arc length of the Catmull-Rom spline. The *shader* processes masks with alpha clipping (`uCorteAlfa`) to shape the jagged edges of the *papel picado* and discard transparent pixels from the scattered petals, drastically reducing the geometric load.

To avoid geometric overlapping or clipping of complex textures, a spatial height grid was implemented in the `ApoyoDulce` class (`ApoyoDulces.h`), which clips each triangle against the corresponding cell, guaranteeing a perfect and natural stacking of the elements.

5.4 Hybrid Lighting and Transmission

The lighting model abandoned the generic directional light from the modeling phase in favor of a hybrid system. This consists of:

- **Directional moonlight:** A faint directional tint modified with ambient occlusion parameters.
- **Point lights:** An array in the *shader* that supports up to 64 simultaneous sources (applied to candles and lanterns) with quadratic attenuation based on a fixed radius.
- **Optical transmission:** An approximate calculation without screen-space refraction, utilizing the index of refraction (`uIor`) and the *Fresnel* calculation to add realism to glass and water objects, reinforcing edges and allowing visibility backward without artifacts.

5.5 Atmospheric Environment (Skybox)

A cubemap for the sky was integrated via the `Skybox` class. The sky's *vertex shader* removes the translation of the view matrix, ensuring the horizon remains static as the player moves. In the *fragment shader* (`cielo.frag`), the pixel's luminance (*luma*) is extracted to dynamically darken the original sunset and blend it into an immersive nocturnal tonality.

6 Architectural Challenges and Cross-Platform Compatibility

Ensuring that the executables and libraries worked smoothly was one of the most challenging aspects. During rigorous integration testing on Arch Linux, macOS, Windows 10, and Linux Mint, the team documented and resolved the following key issues:

1. **Git repository conflicts due to OS-specific configurations:** When collaborating via Git, we frequently overwrote files already validated by a team member by pushing configurations that worked exclusively for macOS/Linux, but broke the compilation on Windows (and vice versa). The implemented solution consisted of unifying the configuration through platform conditionals in the

`CMakeLists.txt` (`if(APPLE)`, `if(WIN32)`), setting up a strict `.gitignore` file to ignore locally generated build directories (`build-xcode/`, `build-windows/`), and being more rigorous with reviews before merging changes.

2. ***Framebuffer* size on high-resolution displays (Retina):** On macOS machines and certain Linux distributions, the image was only rendering in a quarter of the window. We determined that GLFW was returning the size in logical coordinates rather than physical pixels. The solution was to use `glfwGetFramebufferSize` instead of `glfwGetWindowSize` before defining the `glViewport`.

3. **Native texture management without bulky external libraries:** To reduce cross-dependencies that generated linking errors, we opted for a native solution within the Apple ecosystem. By using preprocessor directives (`#ifdef __APPLE__`), the `ImageLoader.cpp` file delegates the loading of PNG and JPEG images to the native `CoreGraphics` and `ImageIO` libraries, additionally executing alpha channel un-premultiplication routines to avoid dark edges in textured cutouts.

4. **Directory separators in the file system:** The *Skybox* textures were failing to load on Windows 10 due to the conflict between slashes (/ vs \). We implemented the modern C++17 ¡`filesystem`¿ standard library for agnostic path management at runtime, which universally resolved the problem.

7 Project Structure

The project incorporates new files and folders intended for the administration of textures and updated *shaders*.

Table 1: New or updated files and folders.

Element	Purpose
`src/render/Skybox.cpp`	Cubemap loading, *luma* manipulation, and spherical sky rendering.
`src/render/Shader.cpp`	Syntactic parser for macro injection (`#define`) and dynamic GLSL compilation.
`shaders/escena.frag`	Lighting algorithms, partial derivatives for reliefs, transmittance, and PBR variables.
`shaders/escena.vert`	Spatial transformation adapted for processing textured instances and color tints.
`assets/textures/`	Base textures, color maps, and metalness/roughness generated procedurally.
`assets/skybox/atardecer/`	Spherical image fragments to process the panoramic environment of the scene.

8 Application Screenshots



Figure 1: General view of the illuminated altar.

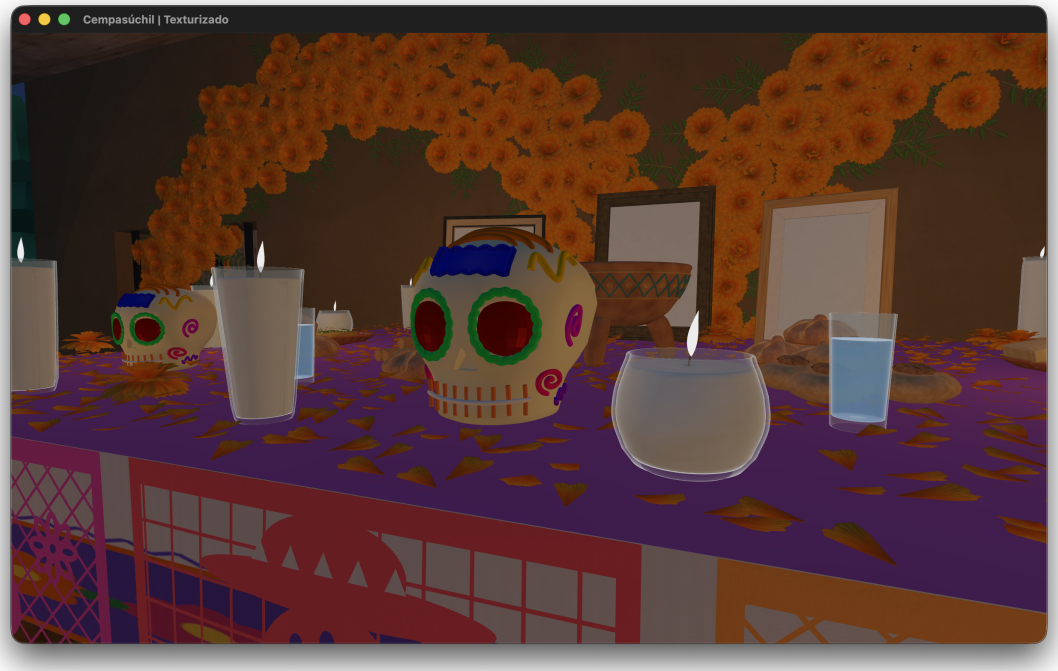


Figure 2: Detail of PBR textures and point light effect on objects.

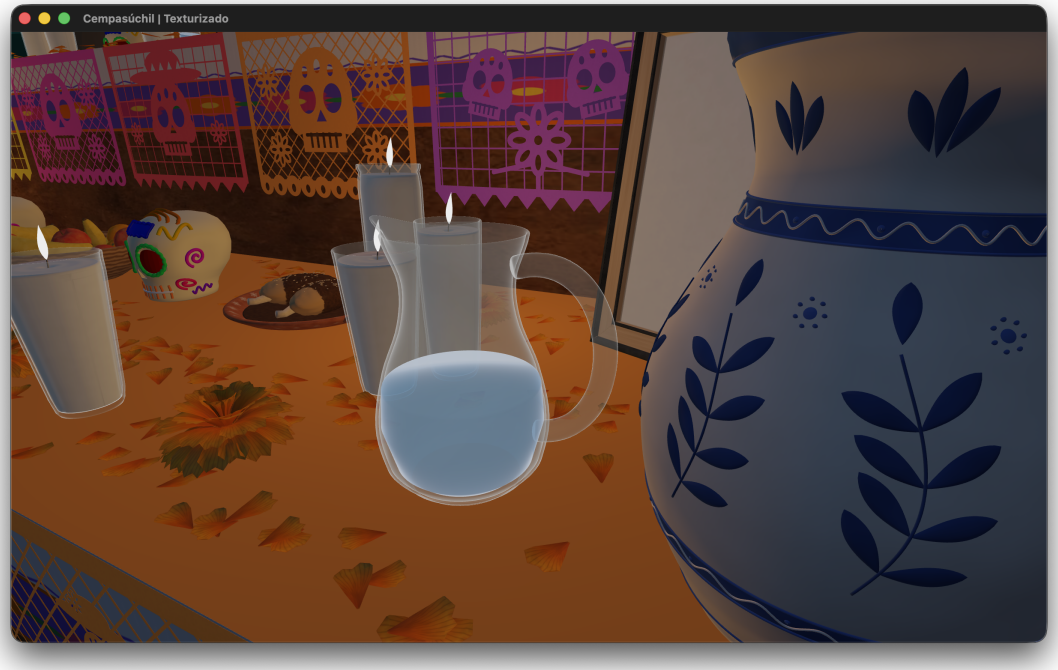


Figure 3: Optical approximation in glass and liquid materials.

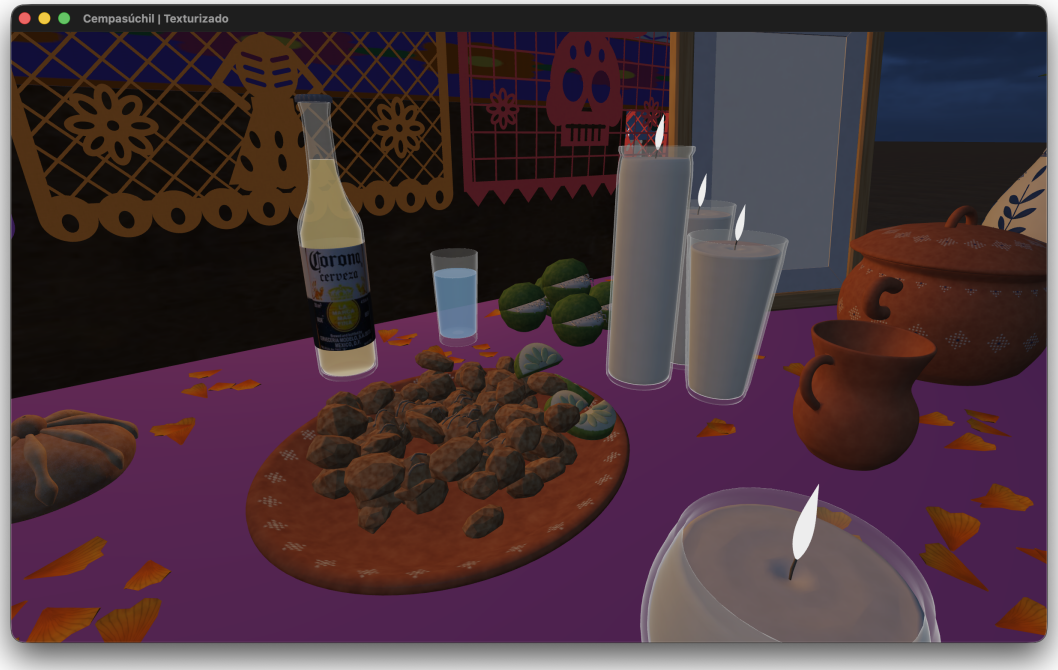


Figure 4: Optical approximation in glass and liquid materials.

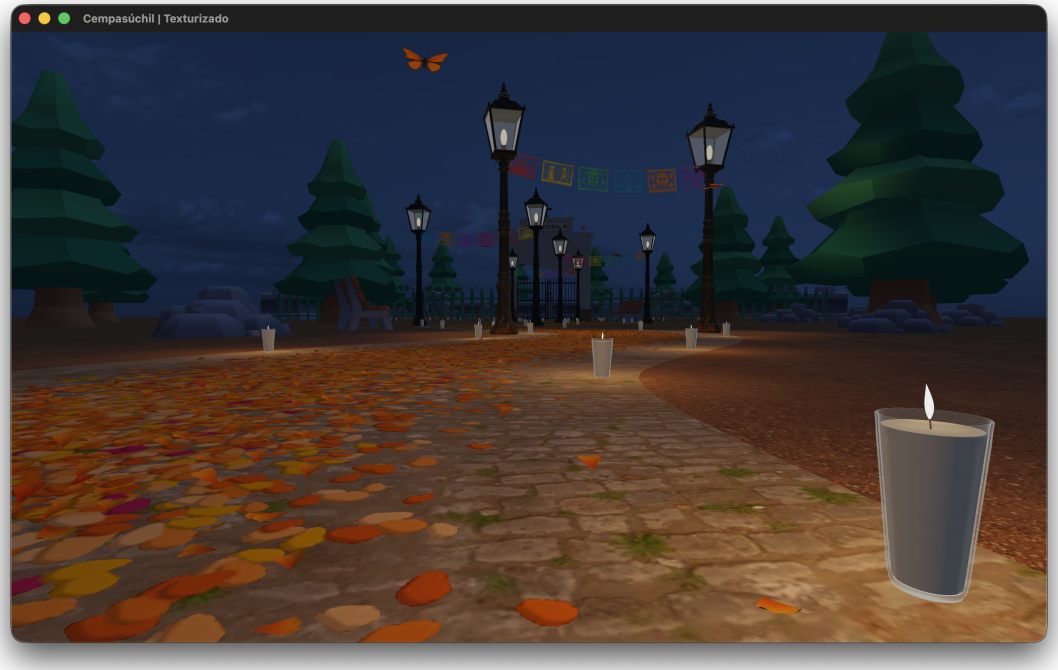


Figure 5: Procedural cobblestone path illuminated by lanterns and candles.

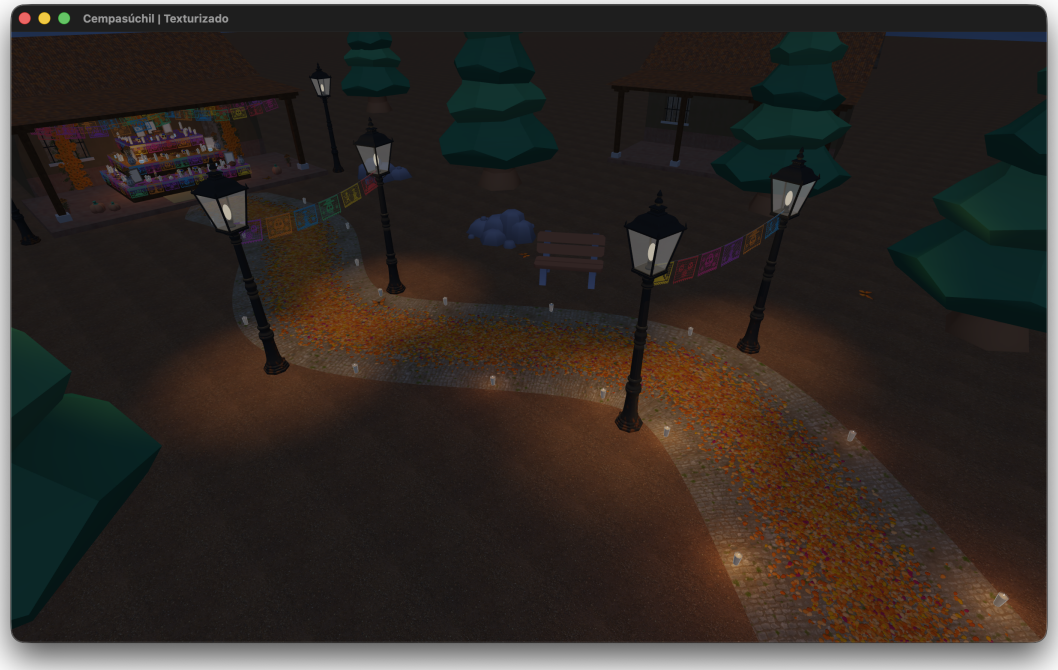


Figure 6: General view of the illuminated path.

9 Compilation and Execution

Code execution has been automated and successfully corroborated by Alexis (Arch Linux), Emmanuel (macOS), Samuel (Windows 10), and Andrés (Linux Mint), ensuring the complete viability of the cross-platform project.

To optimize the workflow and facilitate project review, the team developed native launchers for each operating system. These shortcuts completely automate the build and deployment cycle, standardizing the process under the following features:

- Configure and compile the project via CMake strictly in `Release` mode.
- Update and synchronize the resource directories (`assets` and `shaders`) into the output directory.
- Automatically execute the scene once compilation is finished without running terminal processes.
- Keep the console hidden during the process to offer a clean graphical experience.
- Redirect and write all technical output, warnings, and diagnostics into the `logs/` folder located at the root of the project.

9.1 Launchers by Platform

To visualize the scene, simply use the launcher corresponding to your operating system, which can be found in the `ejecutables` directory:

- **macOS:**
  Launcher packaged as a native application.
  `ejecutables/macOS/TexturizadoCempasuchil.app`
- **Windows:**
  Launcher based on VBScript for silent execution without a console.
  `ejecutables/windows/TexturizadoCempasuchil.vbs`
- **Linux:**
  Standard desktop launcher for Linux environments.
  `ejecutables/linux/TexturizadoCempasuchil.desktop`

NOTE: For further details on the internal workings of these automated launchers, to review the exact dependencies of each system, or to check the location of the log and diagnostic files, it is highly recommended to read the `README.md` file located at the `ejecutables/README.md` path.

10 Results and Conclusions

The texturing and lighting phase significantly elevated the visual impact and immersion of the Day of the Dead environment. However, implementing the hybrid lighting model presented a major performance challenge for the team. By integrating dynamic light sources, we detected severe bottlenecks

because some models exported from Blender had an excessive polygon density; the graphics engine was calculating lighting per fragment over an unnecessary amount of triangles, saturating the processing pipeline. The implemented solution was to optimize these models, drastically reducing their triangle count (retopology and decimation) while retaining a visual appearance practically identical to the original thanks to the detail provided by the normal maps.

The use of procedurally crafted PBR textures, coupled with normal calculations derived directly in the *fragment shader*, demonstrated an excellent balance between photographic quality and algorithmic optimization. Likewise, the cross-platform architectural design was crucial in successfully overcoming technical barriers related to native libraries, file systems, repositories, and high-resolution interfaces.

Proper alpha mapping to reduce polygons (as seen in the petal carpet), geometric optimization of the meshes, and the simulated refraction system allow us to affirm that the delivery satisfactorily meets the proposed visual parameters. For the final phase, the robust and optimized technical environment built so far will serve as an ideal foundation to seamlessly integrate particle systems and animations.

11 References

- Blender Foundation. (2026). *Blender 5.2*. `https://www.blender.org/`
- Khronos Group. (2021). *glTF 2.0 Specification*. `https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html`
- Khronos Group. (2010). *OpenGL 3.3 Core Profile Specification*. `https://registry.khronos.org/OpenGL/specs/gl/glspec33.core.pdf`